

ALGORITHMS PROJECT

“10 – MISSING INTEGER”

Pseudo code “Recursive”:

Pseudo-code “recursive”

ALGORITHM Merge(ARR1, L1, ARR2, L2, ARR3)

$i \leftarrow 0$

$j \leftarrow 0$

$k \leftarrow 0$

WHILE $i < L1$ AND $j < L2$

IF $ARR1(i) < ARR2(j)$

$ARR3(k) \leftarrow ARR1(i)$

$i \leftarrow i + 1$

ELSE

$ARR3(k) \leftarrow ARR2(j)$

$j \leftarrow j + 1$

$k \leftarrow k + 1$

WHILE $i < L1$

$ARR3(k) \leftarrow ARR1(i)$

$i \leftarrow i + 1$

$k \leftarrow k + 1$

WHILE $j < L2$

$ARR3(k) \leftarrow ARR2(j)$

$j \leftarrow j + 1$

$k \leftarrow k + 1$

Pseudo-code "recursive"

```
ALGORITHM MergeSort(A, n)
  IF  $n \leq 1$ 
    RETURN
  leftSize  $\leftarrow n / 2$ 
  rightSize  $\leftarrow n - \text{leftSize}$ 
  MergeSort(A, leftSize)
  MergeSort(A + leftSize, rightSize)
  CREATE array Temp(n)
  Merge(A, leftSize, A + leftSize, rightSize, Temp)

  FOR  $i \leftarrow 0$  TO  $n-1$ 
     $A[i] \leftarrow \text{Temp}[i]$ 
```

Pseudo-code "recursive"

```
ALGORITHM MissingInteger(A, n, missing, index)

  IF  $\text{index} > n$ 
    RETURN missing

  IF  $A[\text{index}] \leq 0$  or  $A[\text{index}] < \text{missing}$ 
    RETURN MissingInteger(A, n, missing, index + 1)

  IF  $A[\text{index}] = \text{missing}$ 
    RETURN MissingInteger(A, n, missing + 1, index + 1)

  IF  $A[\text{index}] > \text{missing}$ 
    RETURN missing

  ALGORITHM Solution(A, n)
    MergeSort(A, n)
    RETURN MissingInteger(A, n, 1, 0)
```

Implementation “Recursive”:

```
#include <stdio.h>
void merge( int ARR1[] , int LARR1 , int ARR2[] , int LARR2 , int ARR3[]){
    int i = 0 , j = 0 , z = 0;
    while( i < LARR1 && j < LARR2 ){
        if( ARR1[i] < ARR2[j]){
            ARR3[z++] = ARR1[i++];
        }
        else{
            ARR3[z++] = ARR2[j++];
        }
    }
    while(i<LARR1){
        ARR3[z++] = ARR1[i++];
    }
    while(j < LARR2){
        ARR3[z++] = ARR2[j++];
    }
}

void merge_sort(int array[] , int len){
    if (len <= 1) return ;
    int ls = len/2; int rs = len - ls;
    merge_sort( array , ls );
    merge_sort( array + ls , rs );
    int tmp[len] ;
    merge( array , ls , array + ls , rs , tmp );
    for(int i = 0 ; i < len ; i++)
        array[i] = tmp[i] ;
}

int missingInt(int A[], int n , int missing , int index){
    if(index >= n)
        return missing;
    if(A[index] <= 0 || A[index] < missing)
        return missingInt(A,n,missing,index+1);
    if(A[index] == missing)
        return missingInt(A,n,missing+1,index+1);
    if (A[index] > missing)
        return missing;
}

int solution(int A[] ,int n){
    merge_sort( A , n );
    return missingInt(A,n,1,0);
}
```

Recurrence Relation "Recursive":

Recurrence Relation "recursive algorithm"

-Recurrence of merge-sort():

$$T(n) = 2T(2n) + n$$

- Master's Method:

$$a = 2$$

$$b = 2$$

$$n^{\log_b(a)} = n^{\log_2(2)} = n$$

$$f(n) = n$$

$$f(n) = O(n)$$

since:

$$O(n) = O(n)$$

$$\text{so case 2: } T(n) = O(n \cdot \log(n))$$

-Recurrence of missingInt():

$$M(n) = M(n-1) + c$$

using iterative method :

$$M(n-1) = M(n-2) + c$$

$$M(n-2) = M(n-3) + c$$

$$M(n-3) = M(n-4) + c$$

.

.

.

$$M(1) = M(0) + c$$

$$\text{so } M(n) = M(1) + \sum_{i=1}^n c$$

$$\rightarrow M(1) + c \cdot n \rightarrow M(1) + O(n) \rightarrow O(n)$$

$$\text{so } M(n) = O(n)$$

Overall Complexity of solution():

$$T(n) = O(n \cdot \log(n)) + O(n)$$

$$T(n) = O(n \cdot \log(n))$$

Pseudo code "Non-Recursive":

Pseudo-code "non-recursive"

ALGORITHM Insertion Sort(A, n)

FOR $i \leftarrow 1$ TO $n-1$

$key \leftarrow A[i]$

$j \leftarrow i - 1$

 WHILE $j \geq 0$ AND $A[j] > key$

$A[j + 1] \leftarrow A[j]$

$j \leftarrow j - 1$

$A[j + 1] \leftarrow key$

ALGORITHM MissingInt(A, n)

 Insertion Sort(A, n)

$missing \leftarrow 1$

 FOR $i \leftarrow 0$ TO $n-1$

 IF $A[i] \leq 0$

 CONTINUE

 IF $A[i] = missing$

$missing \leftarrow missing + 1$

 ELSE IF $A[i] > missing$

 BREAK

 RETURN $missing$

Implementation “Non-Recursive”:

```
#include <stdio.h>

void insertionSort(int arr[], int n){
    int key, j;
    for (int i = 1; i < n; i++){
        key = arr[i];
        j = i - 1;
        while (j >= 0 && arr[j] > key){
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

int missingInt(int arr[], int n){
    insertionSort(arr, n);
    int missing = 1;
    for (int i = 0; i < n; i++){
        if (arr[i] <= 0)
            continue;
        if (arr[i] == missing){
            missing++;
        }
        else if (arr[i] > missing){
            break;
        }
    }
    return missing;
}
```

Recurrence Relation "Non-Recursive":

Recurrence Relation "Non-recursive algorithm"

Recurrence :

$$T(n) = T(n-1) + (n-1) \text{ "worst case"}$$

$$T(n) = T(n-1) + c \text{ "Best case"}$$

using iterative method :

$$T(n-1) = T(n-2) + (n-2) \quad T(n-1) = T(n-2) + c$$

$$T(n-2) = T(n-3) + (n-3) \quad T(n-2) = T(n-3) + c$$

.

.

.

.

.

.

$$T(1) = T(0)$$

$$T(1) = T(0) + c$$

$$\text{so, } T(n) = \sum_{i=1}^n n - i = (n-1)*n/2 = O(n^2)$$

$$T(n) = O(n^2) \text{ "worst case"}$$

$$\text{so, } T(n) = \sum_{i=1}^n c = c*n = O(n)$$

$$T(n) = O(n) \text{ "Best case"}$$

Pseudo code "Non-Recursive 2":

Pseudo-code "Non-recursive 2"

ALGORITHM MissingInteger(A, n, missing, index)

FUNCTION MissingInteger(A, N)

Num $\leftarrow$ 1

WHILE TRUE

Found $\leftarrow$ FALSE

FOR I $\leftarrow$ 0 TO N - 1

IF A(I) = Num THEN

Found $\leftarrow$ TRUE

BREAK

IF Found = FALSE THEN

RETURN Num

Num $\leftarrow$ Num + 1

Implementation “Non-Recursive 2”:

```
#include <stdio.h>
#include <stdbool.h>

int missingInt(int A[], int N)
{
    int num = 1;

    while (1)
    {
        bool found = false;

        for (int i = 0; i < N; i++)
        {
            if (A[i] == num)
            {
                found = true;
                break;
            }
        }

        if (found == false)
        {
            return num;
        }

        num++;
    }
}
```

Recurrence Relation "Non-Recursive 2":

Recurrence Relation

"Non-recursive algorithm 2"

- Recurrence:

Using iterative method :

"Worst Case"

$$T(n) = T(n-1) + n$$

$$T(n-1) = T(n-2) + (n-1)$$

$$T(n-2) = T(n-3) + (n-2)$$

.

.

.

$$T(1) = T(0) + 1$$

$$T(n) = \sum_{i=1}^n n - i$$

$$\text{so, } T(n) = O(n^2)$$

"Best Case"

$$T(n) = T(n-1) + c$$

$$T(n-1) = T(n-2) + c$$

$$T(n-2) = T(n-3) + c$$

.

.

.

$$T(1) = T(0) + c$$

$$T(n) = \sum_{i=1}^n c$$

$$\text{so, } T(n) = O(n)$$

COMPARISON

<i>Criteria</i>	<i>ALGORITHM 1</i>	<i>ALGORITHM 2</i>	<i>ALGORITHM 3</i>
<i>Time complexity</i>	$O(n \cdot \log(n))$	$O(n^2)$ "worst case" $O(n)$ "best case"	$O(n^2)$ "worst case" $O(n)$ "best case"
<i>Space complexity</i>	$O(n)$	$O(1)$	$O(1)$
<i>stability</i>	Not stable	stable	stable
<i>Parallel or serial</i>	Parallel & serial	serial	serial
<i>Type</i>	Recursive	Non-Recursive	Non-Recursive

```
PS G:\Implentations> cd "g:\Implentations\" ; if ($?) { gcc algo_proj_REC.c -o algo_proj_REC } ; if ($?) { .\algo_proj_REC }
```

```
Test 1: A = [1, 3, 6, 4, 1, 2]
```

```
Result: 5 (Expected: 5)
```

```
Test 2: A = [1, 2, 3]
```

```
Result: 4 (Expected: 4)
```

```
Test 3: A = [-1, -3]
```

```
Result: 1 (Expected: 1)
```

```
Test 4: A = [0...100000]
```

```
Result: 5000 (Expected: 5000)
```

```
Execution time: 0.006000 seconds
```

```
PS G:\Implentations> cd "g:\Implentations\" ; if ($?) { gcc algo_proj_Non-REC2.c -o algo_proj_Non-REC2 } ; if ($?) { .\algo_proj_Non-REC2 }
```

```
Test 1: A = [1, 3, 6, 4, 1, 2]
```

```
Result: 5 (Expected: 5)
```

```
Test 2: A = [1, 2, 3]
```

```
Result: 4 (Expected: 4)
```

```
Test 3: A = [-1, -3]
```

```
Result: 1 (Expected: 1)
```

```
Test 4: A = [0...100000]
```

```
Result: 5000 (Expected: 5000)
```

```
Execution time: 0.012000 seconds
```

```
PS G:\Implentations> cd "g:\Implentations\" ; if ($?) { gcc algo_proj_Non-REC.c -o algo_proj_Non-REC } ; if ($?) { .\algo_proj_Non-REC }
```

```
Test 1: A = [1, 3, 6, 4, 1, 2]
```

```
Result: 5 (Expected: 5)
```

```
Test 2: A = [1, 2, 3]
```

```
Result: 4 (Expected: 4)
```

```
Test 3: A = [-1, -3]
```

```
Result: 1 (Expected: 1)
```

```
Test 4: A = [0...100000]
```

```
Result: 5000 (Expected: 5000)
```

```
Execution time: 0.001000 seconds
```

```
PS G:\Implentations>
```